

CSE 2232: User Interface Development Lab

Credits	1
Contact Hours	28
Year	Second
Semester	Second
Prerequisite	CSE 1222: Object Oriented Programming Lab
Course Type	☐ Theory ☒ Laboratory work ☐ Project work ☐ Viva Voce

Motivation

Modern software is experienced largely through its user interface. This course introduces front-end development through HTML, CSS, JavaScript, UI/UX design, and React.js, enabling students to build accessible, responsive, and data-driven web interfaces.

Course Objective

This laboratory course provides hands-on experience in building modern web interfaces. Students will learn HTML, CSS, UI/UX principles, JavaScript, a CSS framework, basic HTTP/REST concepts, and React.js for consuming public APIs.

Course Outcomes (COs), Program Outcomes (POs) and Assessment

CO No.	CO Statement	Corresponding PO	Domain / Level of Learning Taxonomy	Delivery Methods & Activities	Assessment Tools
CO1	To apply UI/UX principles and HTML/CSS to build accessible and well-structured web pages	Design/Development of Solutions (PO3)	Cognitive domain – Level 3	☐ Lecture ☒ Note ☒ Text Book ☐ Audio/Video ☒ Web Material ☒ Lab Manual	☒ CA ☐ Final Exam ☒ Assignment ☒ Notebook ☒ Presentation
CO2	To build interactive and responsive UI components using JavaScript and a CSS framework	Modern Tool Usage (PO5)	Cognitive domain – Level 6	☐ Lecture ☒ Note ☒ Text Book ☐ Audio/Video ☒ Web Material ☒ Lab Manual	☒ CA ☐ Final Exam ☒ Assignment ☒ Notebook ☒ Presentation
CO3	To develop a React.js web application that consumes public APIs using the Fetch API	Modern Tool Usage (PO5)	Cognitive domain – Level 6	☐ Lecture ☒ Note ☒ Text Book ☐ Audio/Video ☒ Web Material ☒ Lab Manual	☒ CA ☒ Final Exam ☒ Assignment ☒ Notebook ☒ Presentation

Assessment and Marks Distribution

Students will be assessed on the basis of their overall performance across all lab exercises, assignments, and the final lab examination.

Component	
Continuous Assessments (CA) — lab exercises and assignments throughout the semester	20%

Component	
Comprehensive Final Exam + Lab Notebook	70%
Class Participation	10%

Lab Course Contents / List of Experiments

Lab 1 — HTML Fundamentals

- Document structure: `<!DOCTYPE>`, `<html>`, `<head>`, `<body>`
- Semantic elements: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`
- Forms: `<input>`, `<select>`, `<textarea>`, `<button>`, associated `<label>`
- Accessibility basics: ARIA roles and attributes, `alt` text for images, meaningful link text, keyboard navigability

Lab 2 — UI/UX Design Principles

- Fundamentals of visual design: hierarchy, alignment, proximity, contrast, and repetition
- Typography in UI: font pairing, size scales, line height, readability
- Colour theory: colour psychology, accessible contrast ratios (WCAG AA), palette tools
- Spacing and layout: whitespace, padding/margin conventions, 8-point grid system
- Wireframing: translating a design brief into low-fidelity wireframes using Figma (or pen-and-paper), then implementing the wireframe as an HTML page

Lab 3 — CSS Fundamentals

- Selectors: element, class, ID, attribute, pseudo-class, pseudo-element
- The box model: margin, border, padding, content
- CSS Flexbox: main/cross axis, `flex-direction`, `justify-content`, `align-items`, `flex-wrap`
- CSS Grid: `grid-template-columns/rows`, `gap`, `grid-area`, responsive grid with `auto-fill/minmax`
- CSS variables and basic theming

Lab 4 — Responsive Design with Bootstrap / Tailwind CSS

- Responsive breakpoints and the mobile-first approach
- **Bootstrap:** grid system, utility classes, pre-built components (navbar, cards, modals, buttons)
- **Tailwind CSS:** utility-first workflow, custom configuration, responsive prefixes (`sm:` , `md:` , `lg:`)
- Choosing the right framework for a project; rebuilding the Lab 2 wireframe using a CSS framework

Lab 5 — JavaScript Basics, DOM and Form Validation

- Variables (`let` , `const`), data types, functions, arrow functions, array methods (`map` , `filter` , `reduce`)
- DOM selection (`querySelector` , `getElementById`) and manipulation (create, update, delete elements)
- Event handling: `addEventListener` , event propagation, `preventDefault`
- Client-side form validation: constraint validation API, custom error messages, regex patterns

Lab 6 — Asynchronous JavaScript, Fetch API, and Browser DevTools

- Synchronous vs. asynchronous execution; the event loop
- Promises: `.then()` , `.catch()` , chaining
- `async / await` syntax and error handling with `try/catch`
- The Fetch API: `fetch()` , reading `Response.json()` , handling HTTP errors
- Browser DevTools: Elements panel (inspect and debug CSS), Network panel (monitor API requests and responses, inspect headers/payloads)

Lab 7 — HTTP and REST: Understanding APIs as a Consumer

- HTTP request/response cycle: methods (GET, POST, PUT, DELETE), status codes (2xx, 4xx, 5xx)
- Anatomy of a URL: scheme, host, path, query parameters, fragments
- JSON structure: objects, arrays, nesting, `JSON.parse()` / `JSON.stringify()`
- Using Postman / Thunder Client / `curl` to manually call a public API

- Reading API documentation: authentication methods (API keys, query params), rate limits, example endpoints
- *Note:* This lab positions students as **consumers** of APIs; building API servers is covered in later courses.

Lab 8 — React.js Basics: Components, Props, State, and Hooks

- Setting up a React app with **Vite** (`npm create vite@latest`)
- JSX syntax and the virtual DOM concept
- Functional components: defining, composing, and rendering
- Props: passing data from parent to child, `PropTypes` for type checking
- State with `useState` : managing and updating local component state
- Side effects with `useEffect` : running code on mount, dependency arrays
- Practice: build a small static card-list UI (no API yet) to solidify component thinking before introducing live data

Lab 9 — Public API Integration: Consuming REST APIs with React

- Connect to a simple public API (e.g., [JSONPlaceholder](#) or [REST Countries](#)) using `useEffect` + `Fetch` inside a React component
- Display a list of fetched items as reusable card components
- Implement loading spinner state and error boundary message
- Practice: passing fetched data down as props, conditional rendering (`&&` , ternary)

Lab 10 — React.js Capstone: Meal Explorer App (Final Project)

Build a complete multi-page Meal Explorer application using the [TheMealDB API](#) (free, no auth required).

App screens and features:

Screen	API Endpoint	Key Feature
Categories Page	<code>categories.php</code>	Display all meal categories as clickable cards with thumbnail and description
Meals by Category Page	<code>filter.php?c={category}</code>	On category click, show all meals in that category as a responsive grid
Meal Details Page	<code>lookup.php?i={mealId}</code>	On meal click, show full details — image, ingredients list, instructions, YouTube link

Technical requirements: - React Router (`react-router-dom`) for navigation between the three screens - Shared `<Navbar>` component with a back button / breadcrumb - `useEffect` + Fetch for all three API calls; loading and error states on each screen - Responsive layout using the CSS framework from Lab 4 (Bootstrap or Tailwind) - Accessible markup: semantic headings, `alt` text on all images, keyboard-navigable cards

Assessment: Students demo the live running app (all three screens functional) and submit the complete lab notebook covering Labs 1–10.

Alternative Final Projects: Students may choose one of the following in place of the Meal Explorer. All alternatives require the same three-screen structure (list → filtered list → detail), React Router, `useEffect` + Fetch, and responsive/accessible UI.

Project	API	Auth Required	Unique Skill Practiced
Country Explorer	REST Countries	No	Navigate to a bordering country from the detail page
Movie Explorer	OMDb API	Free API key	Search-driven entry point instead of category browse
Music Explorer	iTunes Search API	No	<code><audio></code> track preview player

Regardless of project choice, all technical requirements above (React Router, shared Navbar, loading/error states, responsive layout, accessibility) apply.

Textbook

1. Jon Duckett — *HTML and CSS: Design and Build Websites*, Wiley, 2011
(visual reference, still widely used)
2. Marijn Haverbeke — *Eloquent JavaScript* (4th ed.), No Starch Press, 2024
(free online at eloquentjavascript.net)

Recommended Resources

1. **MDN Web Docs** — developer.mozilla.org — authoritative reference for HTML, CSS, and JavaScript

2. **React Official Documentation** — react.dev — interactive tutorials and API reference
3. **Full Stack Open (University of Helsinki)** — fullstackopen.com — free, modern web development curriculum (Parts 0–2 relevant to this course)
4. **The Odin Project** — theodinproject.com — free, project-based HTML/CSS/JS curriculum
5. **Kevin Powell's CSS YouTube Channel** — practical, modern CSS techniques

Free PDF Quick-Reference Books (goalkicker.com)

Topic	Download
HTML5	HTML5 Notes for Professionals (PDF)
CSS	CSS Notes for Professionals (PDF)
JavaScript	JavaScript Notes for Professionals (PDF)
React JS	React JS Notes for Professionals (PDF)

These are community-compiled, free-to-download reference books. Students are encouraged to keep them as quick-reference guides throughout the course.